



# DecimalVariable

|                     |                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------|
| 🕒 Created time      | July 4, 2026 4:58 AM                                                                      |
| ☰ Category          | CommonVariables                                                                           |
| 👤 Last edited by    |  R.A R.A |
| 🕒 Last updated time | July 29, 2026 5:24 AM                                                                     |

```
//Doubles - 4.12, 32, 1.234
//Decimal - 4.12, 32, 1.234

//Int32 - 00000000 00000000 00000000 00000001

decimal bankAccountBalance;

bankAccountBalance = 2.34M;

Console.WriteLine(bankAccountBalance);
```

אנחנו שואלים את עצמנו: למה יש לנו שני סוגים שונים של משתנים שמכילים מספרים עשרוניים, עם נקודה עשרונית?

למה **Double** הוא המועדף למרות שהוא שומר מספרים עשרוניים, כשיש לנו טיפוס בשם **Decimal**?

האם **Decimal** הוא לא השם המושלם למספר שמכיל נקודה עשרונית?

כן, יש לנו שני טיפוסים שונים, אבל יש ביניהם הבדל מהותי:

- **רמת Precision**: ה- **Decimal** הוא המדויק מבין השניים.

הוא מאחסן מספרים עד לרמת דיוק של כ-28 מקומות אחרי הנקודה לפני שמתבצע עיגול. אז למה לא להשתמש בו להכל?

- **עלות מול תועלת:** כמו כל דבר אחר בפיתוח תוכנה, יש עלות לשטח אחסון נוסף שמשמש כל משתנה ומשתנה.

משתנים צריכים להיות מאוחסנים בזיכרון בדיוק כמו כל דבר אחר בקוד שלנו. כשאנחנו שומרים מקום בזיכרון למספר מסוג `Decimal`, אנחנו מקצים לו שטח הרבה יותר גדול.

### לדוגמה – הקצאת זיכרון:

יש לנו `Int32` שמיוצג בזיכרון כך:

```
00000000 00000000 00000000 00000000
```

אפילו אם המספר שלנו הוא 1, אנחנו עדיין מקצים את כל השטח הזה:

```
Int32 - 00000000 00000000 00000000 00000001
```

```
00000000 00000000 00000000 00000001
```

עבור הערך 1.

אנחנו שומרים את כל המקום הזה מראש כי הקצאת הזיכרון מבוססת על **גודל Type**, ולא על הערך עצמו.

אותו עיקרון תקף לגבי `Doubles` ו-`Decimals`.

`Double`. דורש נפח אחסון גדול בהרבה בזיכרון מאשר `Decimal`.

עבור רוב השימושים שלנו, הדיוק האולטרה גבוה הזה לא באמת קריטי.

לכן, לרוב הפעולות נשתמש ב-`Doubles`.

לא רק שהם זולים יותר לאחסון, אלא שגם מהיר יותר למעבד לבצע עליהם פעולות חישוב כמו כפל או חיבור.

### מתי נשתמש ב-`Decimal`?

בתרחישים מאוד ספציפיים, ובעיקר לדבר אחד: **כסף**.

מלבד חישובים פיננסיים או מידות אסטרונומיות מדויקות, אנחנו נמנעים משימוש ב-`Decimal`.

### לדוגמה – בעיות המרה והשמה:

```
bankAccountBalance = 2.34;
```

הקומפיילר לא יאפשר לכתוב את זה כך, מכיוון ש- 2.34 מפורש כברירת מחדל כ- Double, בעוד שהמשתנה שלנו מצפה ל- Decimal.

אנחנו חייבים לזכור שאנחנו לא רוצים לבצע **Type conversions** שגויות.

בניגוד להכנסת Int לתוך Double, המרה משתמעת - **Implicit conversion** בטוחה, אנחנו לא יכולים פשוט להכניס את ה- Double הזה לתוך ה- Decimal.

## למה?

כי למרות שלא נחתוך שום מידע, הערך של ה- Double כבר עוגל טכנית סביב המקום ה--15.

ה- Double פחות מדויק ממה שה- Decimal מצפה לקבל.

אם נבצע את ההמרה, אנחנו למעשה נכניס מידע פגום או פחות מדויק לתוך משתנה שאמור להיות סופר מדויק, וזה אסור.

כדי לפתור זאת בעת כתיבת **Constants** בקוד, אנחנו מוסיפים את האות M בסוף:

```
bankAccountBalance = 2.34M;
```

האות M נבחרה מטעמי נוחות ומייצגת את המילה **Money**.

זוהי האינדיקציה לקומפיילר שהמספר שלפנינו הוא מטיפוס **Decimal**.

במציאות, ברוב הפעמים אנחנו לא כותבים ערכים שהם **Hard coding**.

אנחנו מקבלים את המידע ממשתמש או מ **Database**, ובמקרים אלה המידע כבר יגיע בטיפוס הנכון.

אבל אם אנחנו מקלידים את הערך ידנית, עלינו להשתמש ב-M כדי למנוע מהקומפיילר לפרש אותו כ- **Double**.

## ה-Math Operations

רוב הפעולות המתמטיות כמו פונקציות מתקדמות לא מתוכננות לעבוד ישירות על **Decimals**.

אנחנו רגילים לחשוב על פעולות שמחזירות **Doubles** מכיוון שכאמור, הרבה יותר זול ויעיל לחשב אותן כך.

כל פונקציות המתמטיקה המובנות של מיקרוסופט כמו מחלקת **Math** עובדות עם **Doubles** ולא עם **Decimals**.

אם אנחנו כבר משתמשים ב- **Decimal**, סביר להניח שמדובר בכסף.

וכמה פעמים ביום אנחנו כבר צריכים להוציא שורש ריבועי מחשבון העו"ש שלנו?

בדרך כלל, פעולות על כסף מסתכמות בחיבור וחיסור.

זו הסיבה שאין המון פונקציות מתמטיות מובנות סביב `Decimal` – הוא פשוט יקר מדי חישובית, ולכן לרוב צרכי המתמטיקה אנחנו נעזרים ב- `Double`.